

UNIVERSITY OF ECONOMICS HO CHI MINH CITY
COLLEGE OF TECHNOLOGY AND DESIGN
INSTITUTE OF INTELLIGENT & INTERACTIVE TECHNOLOGIES

UEH CREATIVE ROBOT CONTEST 2026
SIMULATION ROUND TECHNICAL REPORT
SENSOR-BASED AUTONOMOUS DRIVING
FOR A TURTLEBOT3 WAFFLE ROBOT

Participant: Nguyen Hoang Phuoc

Student ID: 31231021201

Individual competition

Ho Chi Minh City, October 2026

ABSTRACT AND CONTENTS

This report describes a compact autonomous driving system for the UEH Creative Robot Contest 2026 simulation round. A Python ROS 2 node estimates the lane from a calibrated camera, recognises traffic controls, checks the swept robot footprint against LiDAR returns, and publishes differential-drive velocity commands. Wheel odometry measures travel and bounds short predictions through missing paint. The controller uses sensor observations rather than stored track coordinates, traffic schedules or simulator ground truth.

The selected evaluation (video_v15) recorded 21.999 m of wheel-odometry path length. Its final state was FOLLOW. This measured distance is accumulated travel, rather than official route completion or an organiser score. The implementation has passed 71 unit and regression tests, ROS fault-injection checks, and an independent clean-container build and launch. The remaining route failure and the absence of overtaking are discussed explicitly.

Section	Subject
1	Task, environment and interfaces
2	System architecture
3	Lane perception and curve handling
4	Signs and traffic lights
5	LiDAR and pedestrian safety
6	Control and runtime parameters
7	Recorded evaluation and results
8	Validation, limitations and improvements
9	AI usage statement and references

1. TASK, ENVIRONMENT AND INTERFACES

The assignment requires autonomous navigation using the robot camera, LiDAR and odometry. The supplied environment combines ROS 2 Humble, Gazebo Classic 11 and a TurtleBot3 Waffle inside an Ubuntu 22.04 Docker image. The development host used Windows with a VMware Ubuntu guest allocated 8 GB RAM and eight virtual CPUs. Development trials used the official track at scale 1.0, with signs, lights, the ramp, the tunnel, the pedestrian and the parked robot present.

The solution is installed as `crc_solution`, separately from `crc_sim`. The organiser starts the simulator before the solution launch command. The application does not spawn or reposition entities, query simulator state, or load track coordinates. Development scripts may choose a test starting pose; these scripts are excluded from the submitted driving package and do not influence runtime decisions.

Interface	Purpose
<code>/camera/image_raw</code>	BGR images for lane and object observations
<code>/camera/camera_info</code>	Camera intrinsic calibration
<code>/scan</code>	Obstacle returns and free-space checking
<code>/odom</code>	Relative pose, heading, speed and travel
<code>/clock</code>	Simulation time for behaviour intervals
<code>/cmd_vel</code>	The sole application motion output

Only one application node publishes motion. A wall-clock timer continues to issue safe commands when simulation time freezes. Sensor source timestamps and receipt times must both remain fresh. Official world, model and configuration assets retained all 19 reference checksums in the development workspace.

2. SYSTEM ARCHITECTURE

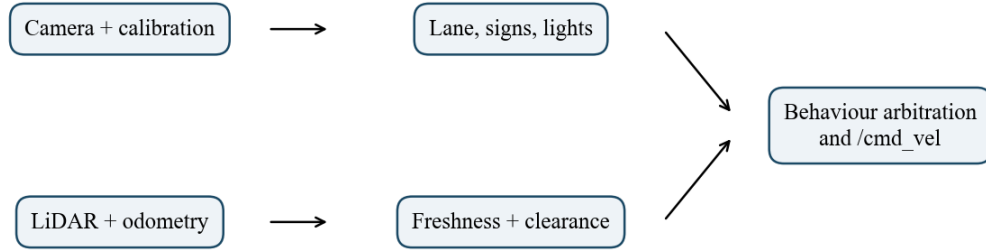


Figure 1. Sensor-to-command architecture, generated by `analysis/submission_figures.py`.

The ROS adapter converts incoming messages, validates their timestamps and keeps the latest accepted sensor samples. Each newly received image is processed once. Lane perception returns a lateral offset, heading estimate, confidence and, when supported, local path curvature. Separate recognisers produce STOP, lamp colour, sign class and pedestrian observations. The behaviour controller gives stopping conditions priority over normal lane following.

Navigation converts the requested lane path into a short candidate arc and checks the actual body and wheel footprint along it. Small steering corrections can increase clearance, while large lane changes remain outside this local planner. The controller then computes forward and angular velocity. Logs associate commands and state transitions with the observations that caused them, so perception failures can be distinguished from watchdog stops.

The main modules are `perception.py`, `boundary.py`, `curves.py`, `navigation.py`, `control.py`, `freshness.py`, `pedestrians.py`, `sign_classifier.py` and `sign_actions.py`. The node adapter in `node.py` owns subscriptions and publishing. The separation allows geometry and behaviour to be exercised without Gazebo, while integration tests cover the ROS message and process lifecycle.

3. LANE PERCEPTION

Lane extraction starts with a grayscale and HSV representation of the camera image. White paint must be sufficiently bright, nearly neutral in saturation, and locally distinct from the surrounding asphalt. A brightness threshold derived from the lower image adapts to changing illumination. This reduces confusion between paint and blue-grey illumination on tunnel walls. A broad bright ramp surface is treated separately from narrow lane markings.

Eleven horizontal image bands supply candidate paint runs. Camera intrinsics, camera height and forward offset project pixels onto an approximate level ground plane. For image row v , depth is estimated as $z = h f y / (v - c_y)$. A pixel u then gives lateral position $y = -(u - c_x) z / f_x$, and forward distance includes the camera offset. These dimensions describe the robot camera; they are not track coordinates.

A pair of paint runs is tested against the 0.35 m lane width. If only one boundary is visible, both possible roles are considered against the previous local centre estimate. A fixed preference for the right edge was removed after it selected the wrong branch on an S bend. Near-ground samples have priority over distant junction markings. A deterministic consensus fit removes isolated transverse stripes before the remaining samples form a local lane regression.

Confidence combines the number and quality of supporting samples with their fitting residual. Low confidence reduces speed. A temporary loss allows only a bounded continuation of the last local arc; persistent loss produces a stop. The ground projection is approximate on slopes, so a clean lane fit on level ground does not guarantee correctness on the ramp.

3.1. CURVES AND MISSING MARKINGS

A straight regression can connect a dashed centre marking to the wrong boundary at a sharp bend. The additional curve fitter evaluates local circular lane corridors in a projected image and requires spatial support for both sides before replacing the regression. A continuous-edge tracer also follows a thin boundary through successive ground-plane rows. It offsets the measured edge along its normal to obtain a candidate lane centre.

The continuous-edge result is used only with strong evidence. A further guard prevents it from reversing a supported lane heading in an oblique straight corridor. Recorded images became regression fixtures: one checks recovery when the robot is outside a visible boundary, and another checks that an adjacent continuous line cannot reverse a supported rightward heading. These tests address different failure modes rather than simply checking an implementation formula.

The short-gap controller is limited by simulation time and odometric travel. It does not replay a stored turn sequence. However, the current curve model can still lose the correct branch when only a distant transverse marking remains visible. A separate `full_v15` trial ended with lane loss after 19.2305 m. The selected recording ends at the time limit without proving complete route coverage. Continuous tracing improves specific bends but is not evidence of complete route robustness.

Mechanism	Evidence or limit
Near-band regression	Rejects distant crossbars and taper noise
Paired curved corridor	Requires paint support on both lane sides
Continuous-edge recovery	Real-image regression fixtures
Gap continuation	At most 4 s and 0.18 m; speed capped at 0.04 m/s
Final failure	Ambiguous later bend remains unresolved

3.2. RECORDED CAMERA OBSERVATIONS

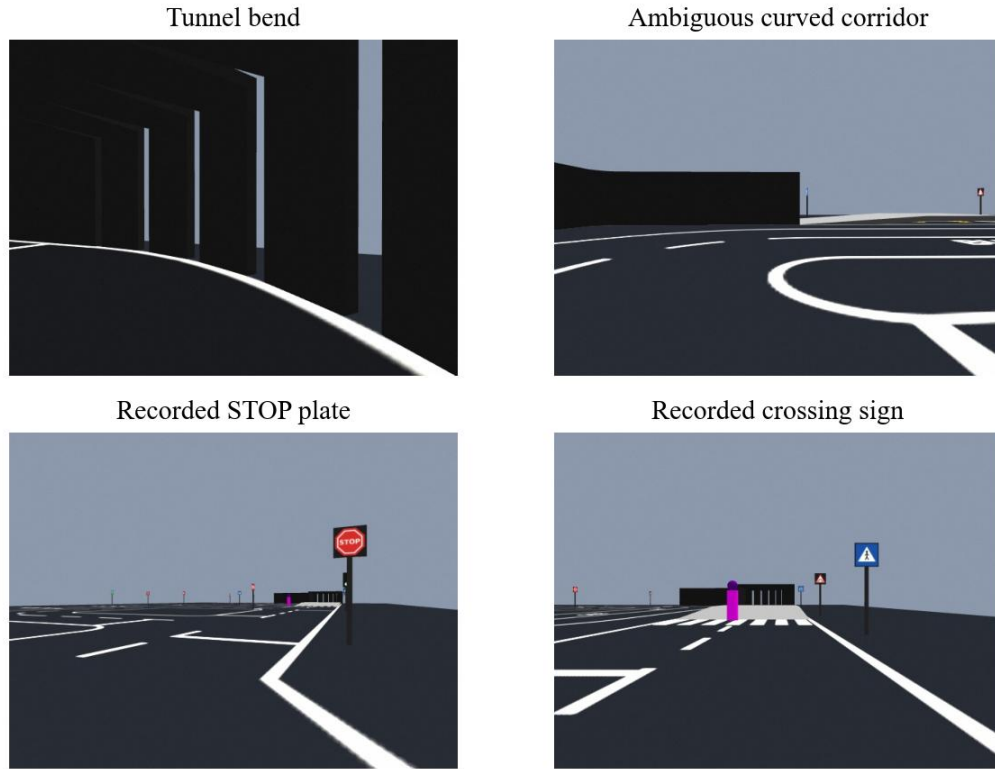


Figure 2. Actual camera frames retained as regression fixtures; images are not ground-truth labels.

The tunnel frame tests illumination and wall-edge ambiguity. The curved corridor frame exposes the failure of a straight line fitted across different paint boundaries. The STOP and crossing images show the supplied sign appearance at realistic camera resolution. Their original files are included with the regression fixtures, and the figure script loads them directly without synthesising scene content.

The figures illustrate observations available to the algorithm. They do not prove that the corresponding manoeuvre was completed without a violation. Behavioural validation additionally uses state transitions and velocities in the selected trial, and the remaining failures are reported separately.

4. SIGNS AND TRAFFIC LIGHTS

STOP detection combines red colour, geometric shape and repeated observations. Three consistent image observations are required near the approach before a stop becomes pending. The controller then waits for measured speed to fall below 0.01 m/s and holds for 2.2 s of simulation time. A distance and visibility-gap condition rearms the sign detector, preventing repeated stops for one plate while allowing a later plate to trigger again.

Traffic-light classification checks coloured compact components inside a bounded, dark, vertically arranged housing. A clipped STOP plate must not be interpreted as a red lamp. Printed sign overlap and housing checks address this ambiguity. Red and yellow observations trigger a stop before the estimated junction entry. Green must be recent and nearby to release a waiting vehicle. Stale entry estimates are discarded after a detection gap, and odometry is used only for the short approach to the observed lamp plane.

Eight appearance templates are packaged with the solution, covering STOP, pedestrian crossing, bus stop, ramp, tunnel, uneven road, highway entry and highway exit. The classifier reads these small local assets and never accesses simulator model textures at runtime. Confirmed caution signs temporarily cap speed at 0.10 m/s over an odometric extent. Highway signs update a state flag; they do not enable an unvalidated overtaking manoeuvre.

Recognition capability and verified behaviour are different claims. Recorded frames support several supplied signs, but changed sign mixes and all eight actions have not been exhaustively validated on unseen layouts. STOP and lamp state transitions in telemetry are useful evidence of implementation execution; they do not independently establish an official traffic score.

5. LIDAR AND PEDESTRIAN SAFETY

LiDAR validation rejects malformed scan geometry and insufficient usable forward measurements. Positive infinity represents a clear ray; NaN or invalid ranges are not silently treated as free space. A three-neighbour median is applied only on continuous surfaces with small range variation. Depth discontinuities and narrow objects remain eligible obstacle evidence. The stock body and wheel self-return envelope is removed, including ramp-induced near-body returns.

Collision checking sweeps the calibrated body rectangle and wheel footprint along candidate circular arcs. It includes the rear body, because tail swing matters in a narrow bend. The planner prefers the camera path when it is clear and otherwise searches nearby curvatures. This is a local steering correction mechanism. It cannot validate a complete lane change around the blue parked vehicle.

The supplied pedestrian is detected using its purple or indigo appearance, plausible aspect ratio, projected size and ground contact. The crossing guard monitors the estimated road across both lanes. A brief camera occlusion can be bridged by LiDAR returns near the last observed pedestrian; this association expires instead of turning an unrelated static obstacle into a permanent person. The road must remain clear for one second before driving resumes.

The colour cue is specific to the supplied actor. Pedestrian position can be inaccurate at image boundaries and on a non-level surface. The safety response is conservative, and an incorrect lane estimate can cause unnecessary waiting. Generic objects remain covered by LiDAR stopping, but that should not be confused with a general pedestrian recognition system.

6. CONTROL AND RUNTIME PARAMETERS

For a valid target point, local pure-pursuit curvature is computed as $k = 2y / (L^2 + y^2)$. In a measured enclosed corridor the lookahead is shortened from 0.45 m to 0.30 m. When a supported curve supplies its own target geometry, that curvature is used directly. Forward speed is reduced by lane uncertainty, estimated heading deviation and obstacle clearance. Angular velocity is $w = v k$, so the angular limit reduces speed while preserving the checked path curvature.

Parameter	Default	Meaning
max_speed	0.18 m/s	Maximum normal forward speed
max_turn	1.0 rad/s	Angular velocity limit
steering_gain	1.0	Curvature multiplier
stop_distance	0.30 m	Local clearance stop threshold
sensor_timeout	0.8 s	Timestamp and receipt watchdog
lane_grace	4.0 s	Maximum short-gap time
stop_hold	2.2 s	Stationary STOP hold

Motion states include WAIT_SENSORS, STOP_HOLD, OBSTACLE_WAIT, LIGHT_WAIT, CROSSING_WAIT, FOLLOW, LANE_GRACE and LANE_LOST. Sensor failure, traffic holds and occupied crossings suppress motion. Dynamic parameter updates are range-checked and applied without restarting the driver. For a demonstration, changing max_speed from 0.18 to 0.08 m/s changes the forward ceiling while leaving the perception code unchanged.

A single launch command starts the installed solution: `ros2 launch crc_solution run.launch.py`. The node publishes a final zero command on termination. Simulation-time intervals govern behavioural waits; a steady wall-clock timer and source-age checks detect paused clocks, missing messages and repeated stale timestamps.

7. EVALUATION PROTOCOL

The development harness starts the official simulator, checks camera, scan and odometry delivery, records a source fingerprint, launches the solution, and archives telemetry and selected images. The selected START trial requests 300 simulation seconds at track scale 1.0. Traffic seed 0 is recorded for reproducibility. A separate no-progress limit can end a trial early; the termination reason is stored explicitly rather than inferred from a successful process exit.

The data set used for this report is stored under `analysis/evidence/video_v15`. Each plot is generated from its `telemetry.csv` by `analysis/submission_figures.py`. The same folder records the source hashes, node interfaces, run configuration and verification log. Accumulated wheel-odometry travel is computed online from successive odometry positions. It can include drift and off-lane travel, so it is neither official completed route distance nor proof that every segment was traversed correctly.

Recorded result	Value
Selected trial	video_v15
Wheel-odometry path length	21.999 m
Requested evaluation duration	300 simulation s
Final logged state	FOLLOW
Control-tick processing median	0.62 ms
Control-tick processing 95th percentile	31.78 ms
Unit and regression checks	71 passed
Full-route completion	Not verified
Official score / collision count	Not measured

7.1. TRAVEL AND VELOCITY RESULTS

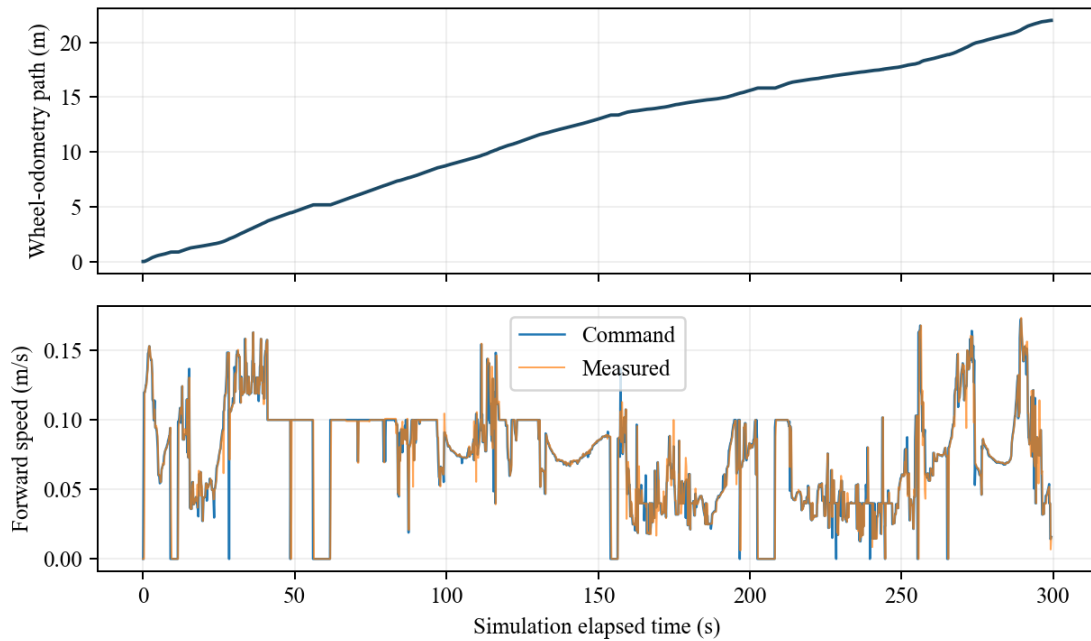


Figure 3. Accumulated odometry and commanded/measured speed from the selected run.

The upper trace shows forward progress and pauses during the recorded interval. The speed trace shows how caution zones, turns and holds reduce the nominal ceiling. A commanded stop and a stable measured speed near zero support the interpretation of a waiting state, but a velocity log alone cannot establish the absence of collision or correct stop-line placement.

The selected recording reached 21.999 m and ended in FOLLOW at the evaluation time limit. Another run of the same source, full_v15, reached 19.2305 m before lane loss. The evidence for both trials is included. Two trials do not establish route-completion reliability, and no mean success rate is claimed. Each evidence folder preserves the complete telemetry and exact source fingerprint.

7.2. LANE ESTIMATES AND BEHAVIOUR STATES

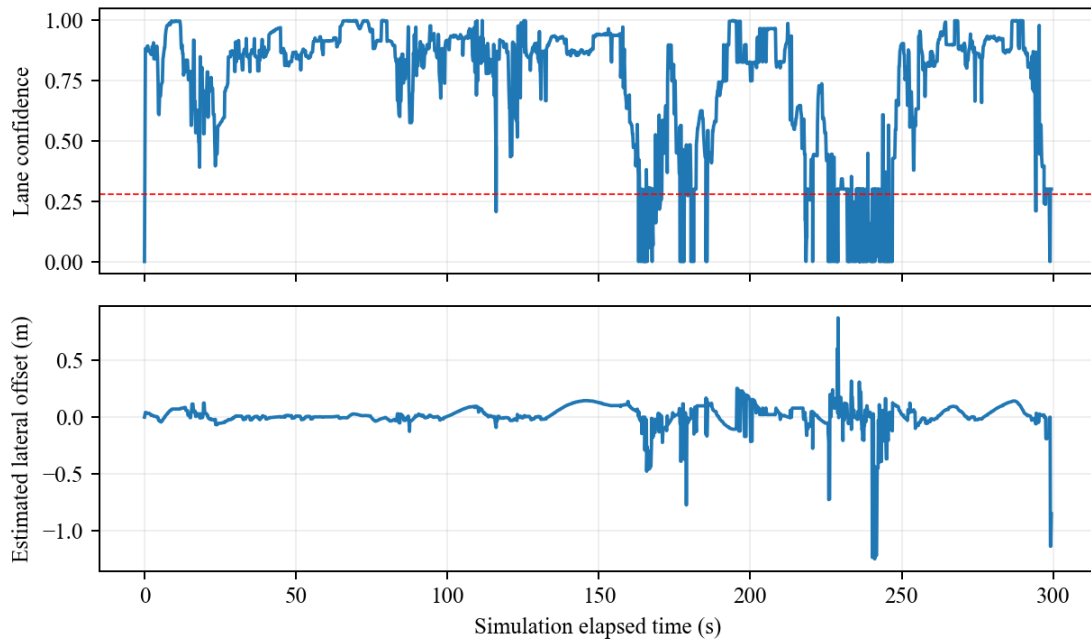


Figure 4. Estimated lane confidence and camera-derived lateral offset.

The RMS of accepted camera lateral estimates is 0.130 m. This is an internal perception statistic. It cannot be substituted for the organiser lane RMS because it is referenced to the estimated lane itself and excludes low-confidence samples. A wrong boundary can produce a confident but incorrect estimate; real-image regression tests were added for precisely this reason.

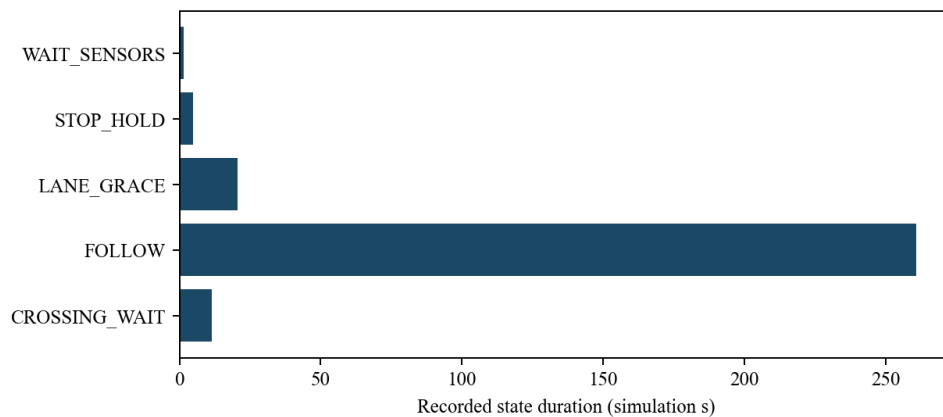


Figure 5. Simulation-time occupancy of recorded behaviour states.

8. VALIDATION AND LIMITATIONS

Seventy-one unit and regression tests pass on Windows and inside the Linux image. Real camera fixtures cover tunnel illumination, ramp masking, ambiguous boundaries and recorded signs. Geometry tests exercise malformed scans, self-return handling and swept collision checking. Behaviour tests cover STOP stationarity, light holds, sign rearming, bounded paint gaps and parameter limits.

ROS integration tests verify motion and live parameter changes, camera or LiDAR loss, invalid ranges, stale source timestamps, future-timestamp recovery, a paused simulation clock and the final stop emitted when the executable receives TERM. A new container copies the package into a clean workspace, builds it and starts the documented launch command. This validates container reproducibility; a separate physical Linux machine was not tested.

The primary limitation is incomplete route coverage. The selected recording ends after 300 simulation seconds while still moving; another run loses the lane at a later bend. Overtaking is absent, and the parked vehicle will invoke conservative stopping if it occupies the checked path. Additional limits include approximate ground projection on slopes, actor-specific pedestrian appearance, unnecessary crossing waits after a wrong lane fit, and incomplete validation under different traffic seeds and object layouts.

Further work should maintain observed boundary identity through odometry, distinguish junction bars from longitudinal paint, and test curve estimation independently of the current robot heading. Overtaking requires an explicit manoeuvre state, observed highway permission and adjacent-lane clearance, followed by sensor-based lane reacquisition. These features should be validated in isolated trials before being enabled in the final launch configuration.

9. AI USAGE STATEMENT AND REFERENCES

OpenAI ChatGPT/Codex was used to assist with system design, implementation, debugging, test generation, analysis scripts and report drafting. Assistance included camera-based lane extraction, traffic-control state logic, LiDAR geometry, ROS watchdogs, regression fixtures and packaging. The project is therefore submitted as AI-assisted work. Generated suggestions were checked against recorded simulator behaviour and automated tests; unsuccessful experiments were not included in the selected runtime revision.

This statement does not imply that every capability has been validated. The limitations and measurements in this report distinguish code checks from successful track traversal. The participant remains responsible for understanding the submitted code, explaining its behaviour and modifying a live parameter during the required competition demonstration. The accompanying simulation-only recording is supporting footage; it does not contain the participant face or English narration required for the assessed six-minute video.

[1] UEH CRC 2026 Organising Committee. UEH_CRC_2026_Simulation_Guideline.pdf, Parts 1-3, supplied competition materials. The later email announces the revised submission deadline; this report does not establish an extension beyond that date.

[2] UEH CRC 2026 Organising Committee. Official simulation pack: ROS 2 Humble, Gazebo Classic 11 and TurtleBot3 Waffle resources, supplied archive.

[3] Project source, tests and recorded telemetry. analysis/evidence/video_v15; analysis/submission_figures.py. Figures 1-5 and all numerical results in this report are reproducible from these files.